

DejaVuGuard User Guide

Itay Cohen

Klaus Havelund

Moran Omer

Doron Peled

RV 2026 Tool Showcase

1 Overview

DejaVuGuard is a runtime verification tool that monitors conversations between a human user and an LLM-based assistant against safety policies written in first-order past-time temporal logic. A separate grounding LLM converts each natural-language message into a structured event containing the relevant predicate instances and canonical object values, and an extended version of the DejaVu engine checks the resulting trace incrementally. If an active policy is violated, the tool blocks the offending message: a violating user message never reaches the assistant, and a violating assistant response never reaches the user.

The tool is part of the repository at https://github.com/itay99988/first_order_rv_guardrails and resides in its `dejavuguard` directory. All commands in this guide are executed from that directory unless stated otherwise.

2 Requirements

The recommended installation uses Docker, in which case Docker with the Compose plugin is enough to start the application and the bundled DejaVu server. To use the chat interface, an OpenRouter API key and chat model must be configured. A grounding model must also be available; it can run locally through Ollama, LM Studio, or vLLM, or remotely through an OpenAI-compatible endpoint including OpenRouter. Running the grounding model locally keeps monitored conversation content on the local machine.

For a development setup without containers, the tool requires Python 3.11 or later, Node.js 18 or later, and Java 17 or later for the bundled DejaVu server. The optional scenario-runner wrapper script documented below uses `uv`; if the package is installed with `pip install -e ".[dev]"`, the same runner can be invoked with plain `python -m scenario_runner`.

3 Installation

3.1 Docker (recommended)

Clone the repository and start the application service and the DejaVu service:

```
git clone https://github.com/itay99988/first_order_rv_guardrails.git
cd first_order_rv_guardrails/dejavuguard
docker compose up --build
```

This builds and starts the DejaVuGuard web application on `http://localhost:8001` and the DejaVu runtime verification server on `http://localhost:8080`. The application service contains both the FastAPI backend and the built React frontend. The SQLite database is stored in the bind-mounted `./data` directory and DejaVu sessions persist in a named volume, so both survive container restarts. A successful start can be confirmed by visiting `http://localhost:8001` or by querying the health endpoints:

```
curl http://localhost:8001/api/health
curl http://localhost:8080/health
```

3.2 Development setup

For development without containers, start the DejaVu server first, then the backend, then the frontend:

```
# Terminal 1: DejaVu runtime verification server
java -jar backend/libs/dejavu.jar --server --port 8080 --storage ./sessions

# Terminal 2: backend
python -m venv .venv
source .venv/bin/activate
# Windows PowerShell: .venv\Scripts\Activate.ps1
pip install -e ".[dev]"
uvicorn backend.main:app --reload

# Terminal 3: frontend
cd frontend
npm install
npm run dev
```

The development frontend is served at `http://localhost:5173` and proxies API requests to the backend on port 8000.

4 Configuration

After the first start, open the Settings page in the web interface. Configure the OpenRouter API key and select a chat model, then configure the grounding provider, its base URL when applicable, and the grounding model, and use the built-in connectivity test. If OpenRouter is also used for grounding, the grounding model can reuse the chat API key or use a separate key. The Settings page also selects whether the chat model or the grounding model generates few-shot demonstrations when a new predicate is created, and exposes the grounding prompt templates for inspection and editing.

All settings can alternatively be provided through environment variables, which act as first-launch defaults before values are saved through the interface. Table 1 lists the variables.

5 Defining Predicates and Policies

Monitoring is configured on the Rules page. A predicate declares an observable fact of the conversation and consists of an identifier used in formulas, a natural language description that the grounding LLM matches against messages, a role stating whether it applies to user or to assistant messages, an arity, and one description per argument slot. For example, a predicate `user_allergic_to` with role `user`, arity one, and the argument description “allergen” captures user statements of a specific allergy. When a predicate is created, the tool generates and validates structured few-shot demonstrations for it using the selected few-shot generation model; if generation fails, the predicate is saved anyway and grounded in zero-shot mode.

A policy is a DejaVu formula over the declared predicates, composed in a text editor with insertion buttons for the available operators. The formula is validated by the DejaVu parser before the policy is saved. Saved policies can be toggled active or disabled from their policy card; chat sessions monitor only enabled policies and only predicates referenced by those policies.

Table 1: Environment variables and their defaults.

Variable	Default	Purpose
OPENROUTER_API_KEY	empty	API key for the chat model
OPENROUTER_MODEL	empty	chat model identifier
OPENROUTER_MODEL_CUSTOM	empty	custom chat model identifier, wins if set
GROUNDING_PROVIDER	ollama	grounding provider: ollama, lmstudio, vllm, custom, or openrouter
GROUNDING_BASE_URL	http://localhost:11434	grounding server base URL
GROUNDING_MODEL	mistral	grounding model name
GROUNDING_API_KEY	empty	separate key for OpenRouter grounding
DATABASE_PATH	./dejavuguard.db	SQLite database location
DEJAVU_URL	http://localhost:8080	DejaVu server base URL
HOST, PORT	0.0.0.0, 8000	backend bind address

When a policy is saved, DejaVuGuard also derives related object positions from variables and comparisons in the formula, and uses those links to provide canonicalization context to the grounding model. Table 2 lists the operators. The built-in proposition `user_turn` holds exactly at user messages and needs no declaration. Since a predicate is only grounded at messages matching its role, formulas that relate user and assistant events must use a temporal operator to refer to the other role’s earlier messages. The allergy policy, for instance, reads

```
forall x . P user_allergic_to(x) -> ! assistant_recipe_contains(x)
```

requiring at every assistant message that no ingredient of a provided recipe equals any allergen the user mentioned at some earlier message.

Table 2: Temporal logic operators in policy formulas.

Operator	Syntax	Meaning
Historically	$H\ p$	p held at every position so far
Previously	$P\ p$	p held at some earlier or current position
Previous step	$@\ p$	p held at the previous position
Since	$p\ S\ q$	q occurred and p has held since then
Timed previously	$P[<=n]\ p$	p held within the last n steps
Timed historically	$H[>n]\ p$	p held at all positions older than n steps
Quantifiers (seen)	$\text{forall } x ., \text{exists } x .$	over values observed in the trace
Quantifiers (full)	$\text{Forall } x ., \text{Exists } x .$	over the unrestricted domain
Connectives	$!, \&, , ->$	negation, conjunction, disjunction, implication
Local rules	$\text{where } r(x) := \dots$	named subformulas within a property

6 Chatting Through the Monitor

The Chat page creates conversation sessions and sends messages through the monitor. Each message bubble shows whether the message passed and offers a details view with the grounding outcome for the active policy predicates evaluated at that message, including extracted mentions and canonical forms, and the per-policy verdicts after that message. When a policy is violated, a banner identifies the policy and its formula, and the offending message or response is blocked.

Note that a violation of an irrevocable formula, such as one of the form $H(\dots)$, persists for the rest of the session; starting a new session resets monitoring.

7 The Scenario Runner

The scenario runner replays scripted user and assistant messages through the same grounding and monitoring pipeline and compares the monitor’s verdicts to expected verdicts recorded in the scenario files. It bypasses the chat LLM because assistant messages are fixed in the scenarios, but it still uses the configured grounding model. On a clean database, it may also need the configured OpenRouter chat model, or the scenario’s `model.few_shot_model`, to generate predicate-specific few-shot examples for newly created predicates. It is the mechanism for reproducing the end-to-end benchmark described in the accompanying benchmark document.

With the backend’s database, the DejaVu server, and the required models available, a scenario set is executed from a local development checkout with

```
python -m scenario_runner --dir scenario_runner/scenarios/recovery_scenario \
    --grounding-provider openrouter --grounding mistralai/ministral-8b-2512
```

The repository’s scenario-runner README uses `uv run python -m scenario_runner`; use that form if you manage the local environment with `uv`. The two override flags select the grounding model for the whole batch without editing the scenario files. Accepted provider names for the runner are `ollama`, `openrouter`, `lm_studio`, `vllm`, and `openai_compatible`. Each batch produces a timestamped directory containing one human-readable log and one JSON log per scenario, a `failures` subdirectory with the mismatched messages of failing scenarios, the rendered grounding prompt per predicate, and Markdown and HTML batch reports. The process exit code distinguishes success, verdict mismatches, setup conflicts, schema errors, and runtime errors, which makes the runner suitable for scripted use. When running against the Docker deployment on macOS, use the wrapper `scripts/run_scenarios.sh`, which stops the application container during the host-side run so that stale bind-mounted SQLite state is not used, and restarts it afterwards.

8 Troubleshooting

If the chat fails immediately, verify the OpenRouter API key and model selection in Settings. If predicates never match, use the grounding health check in Settings and confirm the grounding server is reachable and the model is pulled or selected. Also confirm that the predicate is referenced by an enabled policy; unreferenced predicates are not grounded during chat. If new predicates are saved in zero-shot mode, check the selected few-shot generation model and its API key or local server. If policy creation is rejected, the DejaVu parser message shown in the editor identifies the offending part of the formula; also confirm that every predicate used in the formula exists. If DejaVu is unreachable when a monitor session is created, the chat monitor deliberately fails open and reports policies as passing, so persistent passes together with backend log warnings indicate a connectivity problem with the DejaVu container or Java process. If a policy seems to ignore events of the other role, add a temporal operator as explained above, since predicates are grounded only at messages of their declared role.